

Accurate Predictions on Small Data with a Tabular Foundation Model

Noah Hollmann, Samuel Müller, Lennart Purucker,
Arjun Krishnakumar, Max Körfer, Shi Bin Hoo,
Robin Tibor Schirrmeister, Frank Hutter

Reading Group Presentation

September 18, 2026

The Posterior Predictive Distribution (PPD)

TabPFN / PFNs are trained to **approximate the Posterior Predictive Distribution**.

Definition

Given observed data D and model parameters ϕ , the PPD is the distribution of the label y of a *new* query point x , after integrating out the posterior $p(\phi \mid D)$:

$$\begin{aligned} p(y \mid x, D) &= \int_{\phi} p(y \mid x, \phi) p(\phi \mid D) d\phi \\ &\propto \int_{\phi} p(y \mid x, \phi) p(D \mid \phi) p(\phi) d\phi \end{aligned}$$

- $p(y \mid x, \phi)$ — the **likelihood**: probability of output y given input x and parameters ϕ .
- $p(\phi \mid D)$ — the **posterior** over parameters; expanded by Bayes' rule into $p(D \mid \phi) p(\phi)$.
- $p(\phi)$ — the **prior**: our knowledge/assumptions about ϕ *before* seeing data.

Key point

The PPD is a **posterior-weighted average over all possible models**, so it carries both the uncertainty in the data and the uncertainty in the parameters.

How Is the PPD Usually Obtained?

Goal

Approximate the posterior

$$p(\theta \mid D) \propto p(D \mid \theta) p(\theta)$$

instead of just minimizing the loss at a point.

Main approaches:

- **Variational Inference (VI)**: optimize $q_\phi(\theta)$ via the ELBO

$$\mathcal{L}(\phi) = \mathbb{E}_{q_\phi}[\log p(D \mid \theta)] - \text{KL}(q_\phi \parallel p(\theta))$$

with reparameterization $\theta = \mu + \sigma \odot \epsilon$.

- **MCMC** (e.g. HMC, SGLD): asymptotically exact, but expensive.
- **Laplace Approximation**: Gaussian around the MAP estimate.
- **SWAG / Dropout**: approximate posterior via the SGD trajectory or dropout masks.

Key insight

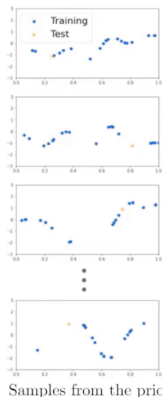
Training learns a *distribution over parameters*, not just a single point estimate.

The High-Level Intuition of PFNs

- TabPFN is a **transformer with weights θ**
 - A **single forward pass** directly approximates $p(y_{n+1} \mid x_{n+1}, \{(x_1, y_1), \dots, (x_n, y_n)\})$
- We optimize θ to minimize the **average cross-entropy loss across datasets**
 - *Across which datasets?*
 - Millions of synthetically generated ones: $\{(x_1, y_1), \dots, (x_{n+1}, y_{n+1})\}$
 - *How do we train it?*
 - Very standard transformer architecture (just drop the positional encoding)
 - Standard supervised learning with SGD

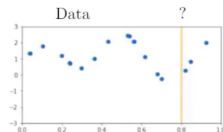


Prior-Data Fitted Networks (PFNs)



Learn a model to
predict test from train
We call this model a
prior-fitted network(PFN)

We have thus meta-learned to
approximate Bayesian inference,
purely by supervised learning
with set-valued inputs



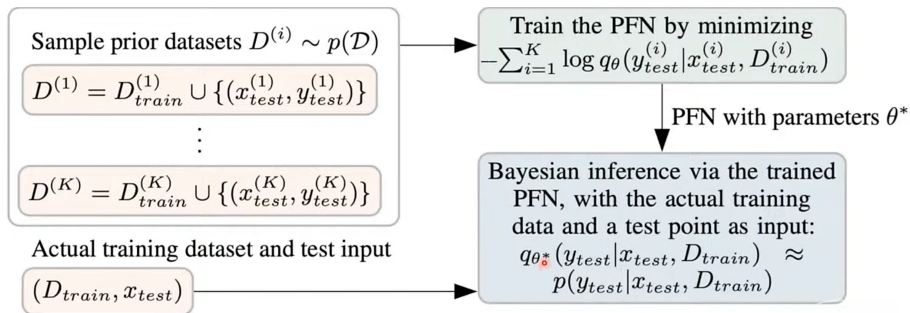
Perform a forward
pass with the PFN



Posterior predictive distribution

Draw datasets from a **prior** (a data-generating mechanism), learn to predict the test point from the train points — **meta-learning Bayesian inference by plain supervised learning**.

PFNs: Training Objective and Inference



Training minimizes the negative log-likelihood on held-out points of *synthetic* datasets; at inference the *real* (D_{train}, x_{test}) is fed in and one forward pass returns $q_{\theta^*}(y_{test} | x_{test}, D_{train}) \approx p(y_{test} | x_{test}, D_{train})$.

Why Does Minimizing the NLL Work? (1/2)

The PFN loss is the **Prior-Data NLL**, ℓ_θ :

$$\begin{aligned}\ell_\theta &= \mathbb{E}_{D \cup \{x, y\} \sim p(\mathcal{D})} [-\log q_\theta(y \mid x, D)] \\&= - \int_{D, x, y} p(x, y, D) \log q_\theta(y \mid x, D) \\&= - \int_{D, x} p(x, D) \int_y p(y \mid x, D) \log q_\theta(y \mid x, D) \\&= \int_{D, x} p(x, D) H(p(\cdot \mid x, D), q_\theta(\cdot \mid x, D)) \\&= \mathbb{E}_{x, D \sim p(\mathcal{D})} [H(p(\cdot \mid x, D), q_\theta(\cdot \mid x, D))]\end{aligned}$$

- $D \cup \{x, y\}$: a dataset of $|D| + 1$ points sampled from the prior $p(\mathcal{D})$.
- $p(\cdot \mid x, D)$: the **true PPD**; $q_\theta(\cdot \mid x, D)$: the model's **approximation**.
- $H(\cdot, \cdot)$ is the **cross-entropy** between the two distributions.

Why Does Minimizing the NLL Work? (2/2)

Minimizing ℓ_θ is minimizing the **expected KL divergence to the true PPD**, up to a constant:

$$\begin{aligned}\mathbb{E}_{x,D} [\text{KL}(p(\cdot | x, D), q_\theta(\cdot | x, D))] \\&= -\mathbb{E}_{x,D} \left[\int_y p(y | x, D) \log \frac{q_\theta(y | x, D)}{p(y | x, D)} \right] \\&= -\mathbb{E}_{x,D} \left[\int_y p(y | x, D) \log q_\theta(y | x, D) \right] + \mathbb{E}_{x,D} \left[\int_y p(y | x, D) \log p(y | x, D) \right] \\&= \mathbb{E}_{x,D} [\text{H}(p(\cdot | x, D), q_\theta(\cdot | x, D))] - \mathbb{E}_{x,D} [\text{H}(p(\cdot | x, D))] \\&= \ell_\theta + C\end{aligned}$$

$C = -\mathbb{E}_{x,D} [\text{H}(p(\cdot | x, D))]$ does not depend on $\theta \implies \arg \min_\theta \ell_\theta = \arg \min_\theta \mathbb{E}[\text{KL}]$.

Algorithm 1: Training a PFN model by Fitting Prior-Data

Input: A prior distribution over datasets $p(\mathcal{D})$, from which samples can be drawn, and the number of samples K to draw

Output: A model q_θ that will approximate the PPD

Initialize the neural network q_θ ;

for $j \leftarrow 1$ **to** K **do**

 Sample $D \cup \{(x_i, y_i)\}_{i=1}^m \sim p(\mathcal{D})$;

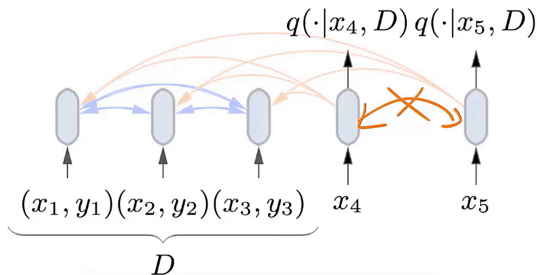
 Compute stochastic loss approximation $\bar{\ell}_\theta = \sum_{i=1}^m (-\log q_\theta(y_i | x_i, D))$;

 Update parameters θ with stochastic gradient descent on $\nabla_\theta \bar{\ell}_\theta$;

end

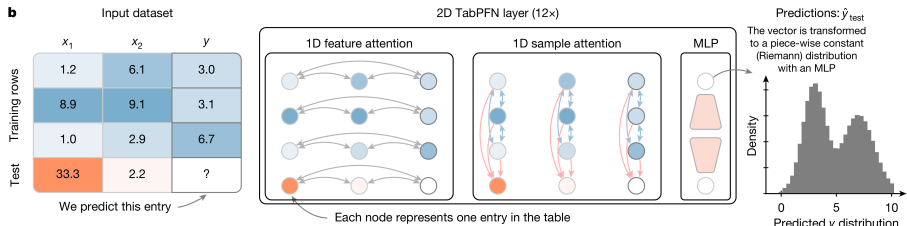
Every step sees a *brand-new* synthetic dataset — no ELBO, no MCMC, only SGD.

A Permutation-Invariant Transformer



- They simply **drop the positional embedding** to obtain permutation invariance.
- They do **not** model correlations between outputs.
 - To draw joint samples, they would draw one at a time and then include it in the input.

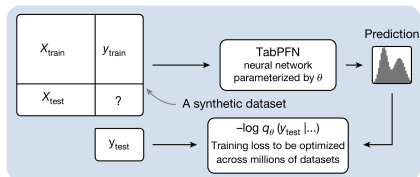
TabPFN v2: an Architecture Designed for Tables



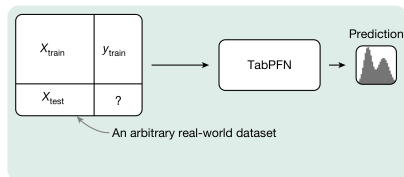
- Every **cell** of the table is its own position $\Rightarrow$ generalizes to more samples *and* more features than seen during training.
- Each of the 12 layers: **1D attention over features** (within a row) $\rightarrow$ **1D attention over samples** (within a column) $\rightarrow$ MLP, each sublayer with residual + half-precision layer norm.
- Test samples attend only to the *training* data, never to each other $\Rightarrow$ train representations can be cached and reused.
- The output vector is turned into a piece-wise constant (Riemann) distribution by an MLP $\Rightarrow$ a full predictive distribution, not a point estimate.

Recall: the Overall Framework

a TabPFN is trained on synthetic data to take entire datasets as inputs and predict in a forward pass



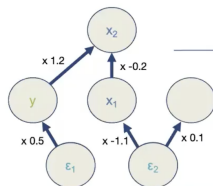
TabPFN can now be applied to arbitrary unseen real-world datasets



- **Pre-training (once):** on synthetic datasets, predict y_{test} from $(X_{train}, y_{train}, X_{test})$ and minimize $-\log q_{\theta}(y_{test} | \cdot)$ across millions of datasets.
- **Usage (per dataset):** feed an arbitrary unseen real-world table in **one forward pass** — no gradient step, no hyperparameter tuning.

The Prior: Structural Causal Models

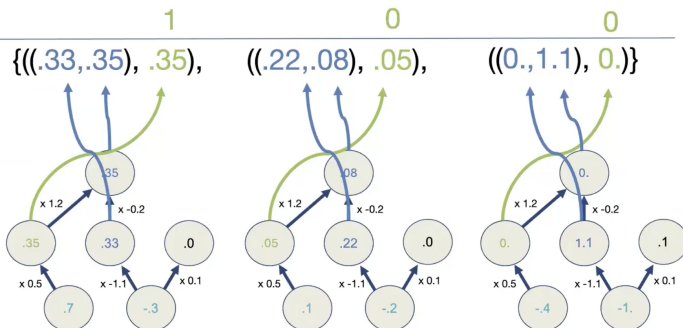
Sample & initialize
a causal graph



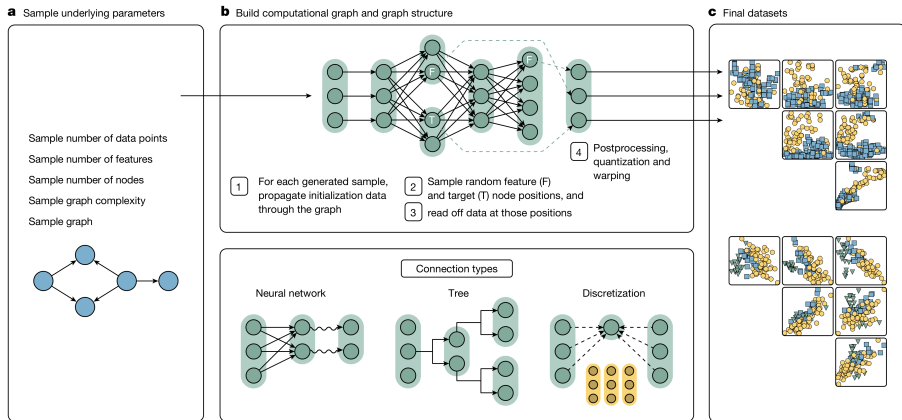
Sample noise
per example
& forward pass

Build dataset:

output > 0.2 ?



How the Prior Is Generated



a Sample high-level hyperparameters (# data points, # features, # nodes, graph complexity). **b** Build a structural causal model: (1) propagate initialization noise through the computational graph, (2) sample feature (F) and target (T) node positions, (3) read off the data at those positions, (4) post-process, quantize and warp. **c** The resulting datasets.

Training Details

1. Training-set size and computational cost

- $\approx 2,000,000$ SGD steps, **batch size 64 datasets** $\Rightarrow \approx 130,000,000$ synthetic datasets per model ($\approx 100M$ unique causal structures).
- One training run: $\approx$ **2 weeks on one node with $8\times$ NVIDIA RTX 2080 Ti** — affordable for an academic lab, and done *once*.
- Inference is then a single forward pass: **2.8 s** (classification) / **4.8 s** (regression) on a 5-year-old consumer GPU.

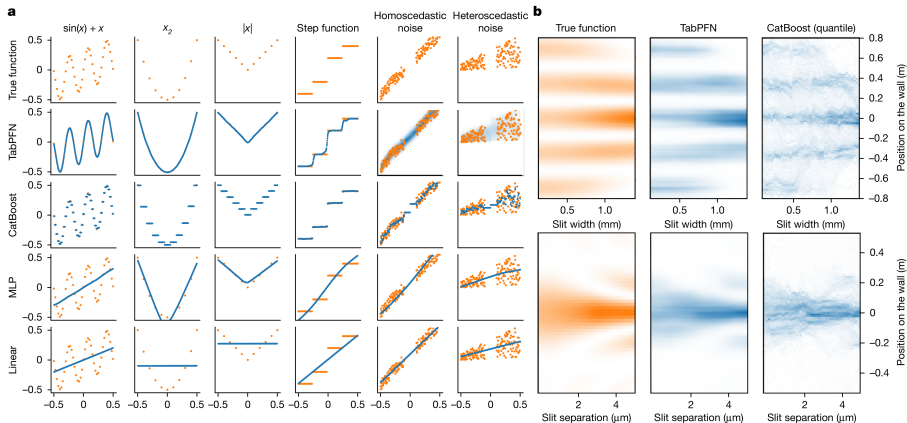
2. Boundaries of the prior

- # training samples per dataset: uniform up to **2,048**; fixed validation size **128**.
- # features: Beta($k=0.95$, $b=8.0$) linearly scaled to the range **1–160**.
- Table size capped at **75,000 cells** (fewer samples when many features) to avoid memory peaks.
- Graph: # nodes $N \sim \mathcal{U}_{\log}(a, b)$, redirection probability $P \sim \Gamma(\alpha, \beta)$ — larger N = bigger graphs, smaller P = denser graphs.
- Prior hyperparameters chosen by **random search** on a dev set $\Rightarrow$ the PPD is a Bayesian ensemble over that hyperparameter space.

3. Model and optimization settings

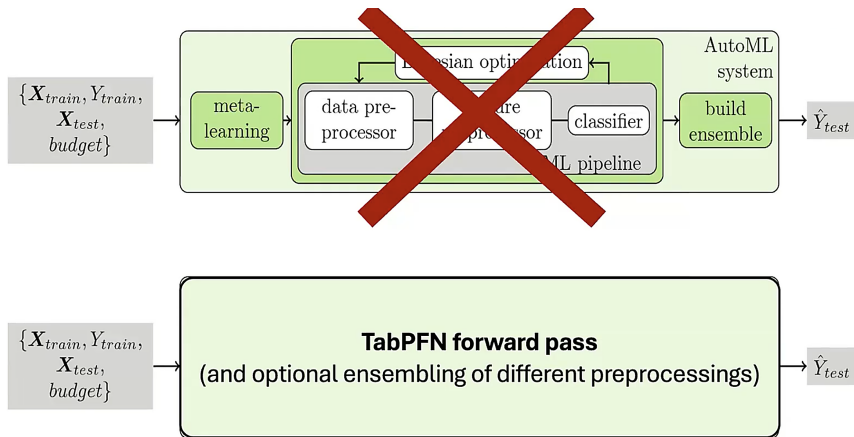
- Transformer encoder variant, **12 two-dimensional layers**; one position encodes *two* features of one example by default.
- Random feature embeddings added before layer 1; multi-query attention + KV caching for test samples.
- Search space of **6 architectures** (classification) / **5** (regression).
- **Adam** with linear warmup and cosine annealing; learning rate picked from $[10^{-4}, 5 \times 10^{-4}]$.

The Behaviour of TabPFN



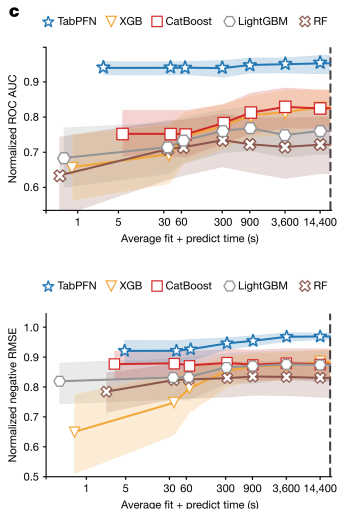
a Orange = ground truth, blue = model prediction. TabPFN fits smooth, piece-wise and noisy functions, and — unlike the baselines — keeps sensible behaviour under homo-/heteroscedastic noise. **b** It models *distributions* over outputs out of the box: the double-slit intensity pattern recovered from the positions of 1,000 photons.

Simplicity: One Forward Pass, No Pipeline



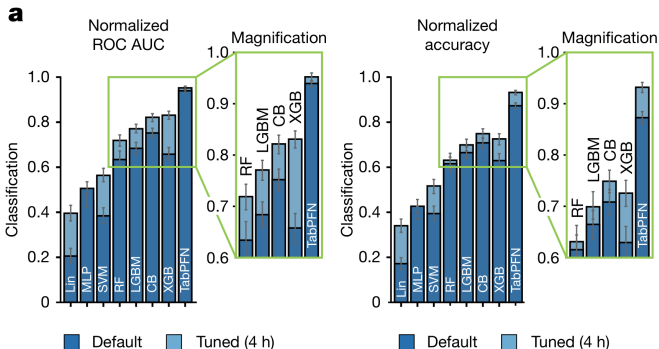
No meta-learning stage, no Bayesian optimization loop, no per-dataset pipeline search — just **one forward pass** (optionally ensembled over different preprocessings).

Efficiency: Accuracy per Second



- x-axis: average fit + predict time; baselines are given up to **4 hours** of tuning.
- TabPFN's **default** configuration already sits above every tuned baseline, on both classification (ROC AUC) and regression (negative RMSE).
- Speed-up at matched performance:
 - **5,140×** for classification
 - **3,000×** for regression
- The gap does *not* close as the baselines get more budget — it is a different operating point, not a head start.

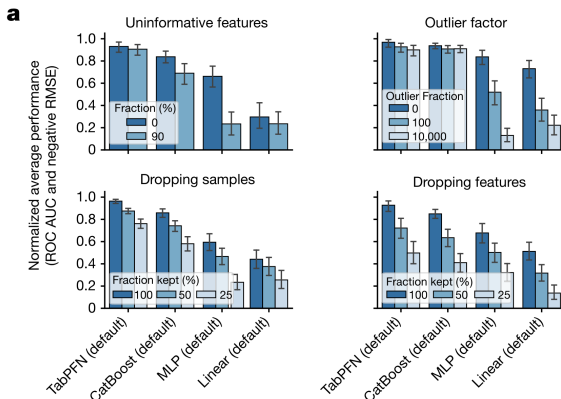
Performance vs. State-of-the-Art Baselines



Classification, up to 10,000 samples and 500 features; scores normalized per dataset, 95% CIs.

- ROC AUC: TabPFN beats CatBoost (strongest *default* baseline) by **0.187** (0.939 vs. 0.752) by default, and by **0.130** (0.952 vs. 0.822) when both are tuned 4 h.
- For regression, normalized RMSE improves by **0.051** (default) and **0.093** (tuned).
- **Default TabPFN outperforms every tuned baseline.**

Robustness Across Modified Datasets



Datasets are perturbed by adding uninformative features, injecting outliers, dropping samples and dropping features (scores normalized per dataset, one normalization shared across all modifications of an experiment).

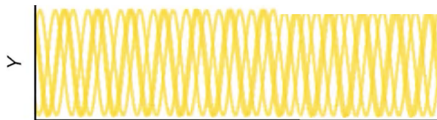
- TabPFN degrades **no faster** than CatBoost, MLP or a linear model under every modification.
- It reproduces the accuracy of **default CatBoost using only half the training samples**.

Finetuning: Customizing the Model to Your Data

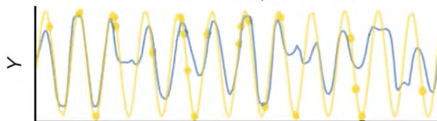
- TabPFN is **pretrained** on large amounts of data
 - 130m synthetic datasets
- TabPFN can be **finetuned** to your own type of data
 - This can make it dramatically better
 - In contrast to traditional XGBoost etc., which always learn from scratch

- Prediction
- Ground truth
- Training sample

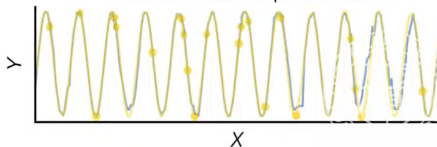
Fine-tuning data



Default TabPFN predictions



Finetuned TabPFN predictions



Conclusion

- Reframes tabular prediction as **in-context learning**: a transformer pre-trained *once* on $\approx 100\text{M}$ synthetic datasets drawn from structural causal models approximates the posterior predictive distribution in a single forward pass.
- On tables with up to 10,000 samples and 500 features it beats *tuned* GBDT baselines in seconds rather than hours, and stays robust to uninformative features, outliers and missing values.

Is it worth reading? Yes.

- But for learning purpose, the theory is cited rather than developed.
- For the mechanism, read the foundations first: *Transformers Can Do Bayesian Inference* (Müller et al., ICLR'22) and TabPFN v1 (Hollmann et al., ICLR'23).

Is it worth implementing? Yes.

- TabPFN is very popular on GitHub and actively maintained.
- Pre-trained checkpoints *and* a hosted API are released — usable with no GPU and no training of your own.